

CASO – DESARROLLO GUIADO POR PRUEBAS (TDD)

Sistema de Gestión de Pedidos en Java 17 + JUnit 5 · Calidad y Pruebas de Software · VIII ciclo

Integrantes	
Sección / Grupo	
Enlace al repositorio	
Fecha	

1. Logro de la actividad

Al finalizar el caso, el estudiante implementa en Java el cálculo del total de un pedido aplicando el ciclo Red – Green – Refactor: escribe cada prueba unitaria con JUnit 5 antes del código, cubre casos normales, valores límite, redondeo y validaciones, y evidencia el proceso con su historial de commits.

1. Aplicar el ciclo Red – Green – Refactor un requisito a la vez.
2. Escribir pruebas con JUnit 5 (@Test, @ParameterizedTest, @CsvSource, assertThrows).
3. Manejar montos de dinero con BigDecimal y una regla de redondeo explícita.
4. Aplicar valores límite (Semana 08) al porcentaje de descuento y a las cantidades.
5. Refactorizar con seguridad usando las pruebas como red de protección.

2. Recordatorio: el ciclo TDD

Fase	Qué haces
● Red	Escribes una prueba para un comportamiento que aún no existe y confirmas que falla. En Java, que el proyecto no compile porque la clase o el método no existen también cuenta como Red.
● Green	Escribes el código mínimo para que la prueba pase. No adelantes funcionalidad que ninguna prueba pida.
● Refactor	Mejoras nombres, eliminas duplicación y ordenas el código sin cambiar su comportamiento. Todas las pruebas deben seguir en verde.

3. Descripción del caso

Una tienda en línea peruana necesita calcular el total de un pedido a partir de los productos seleccionados, sus cantidades, un cupón de descuento opcional y el IGV (18%).

Requisitos funcionales

ID	Requisito	Detalle
RF1	Subtotal	Suma de precio $\times$ cantidad de cada producto. Un pedido sin productos tiene subtotal S/ 0.00.
RF2	Descuento	Si hay cupón, se resta su porcentaje (0 a 100) al subtotal. Ej.: subtotal S/ 100.00 con cupón de 10% $\rightarrow$ S/ 90.00. Sin cupón, el porcentaje es 0.
RF3	IGV	Se calcula el 18% sobre el monto después del descuento (base imponible).
RF4	Total	Total = base imponible + IGV. Ej.: S/ 90.00 + S/ 16.20 = S/ 106.20.
RF5	Redondeo	Todos los montos se devuelven con 2 decimales, redondeando 0.005 hacia arriba: <code>setScale(2, RoundingMode.HALF_UP)</code> .
RF6	Validaciones	Precio negativo, cantidad ≤ 0 o descuento fuera de 0 – 100 $\rightarrow$ <code>IllegalArgumentException</code> con un mensaje claro.

Importante: los montos se manejan con `BigDecimal`, nunca con `double`. Créalos desde un `String`: `new BigDecimal("25.50")`. Con `double`, `0.1 + 0.2` da `0.30000000000000004` y las pruebas de dinero fallan por centésimas.

Cuidado con `assertEquals` y `BigDecimal`: `equals` compara también la escala (cantidad de decimales). Para Java, 16.2000 y 16.20 no son iguales. Por eso el RF5 exige devolver siempre 2 decimales, y las pruebas esperan valores como `new BigDecimal("16.20")`.

Firma de las clases a construir

```
public record Producto(String nombre, BigDecimal precio, int cantidad) { }

public class CalculadoraPedido {
    BigDecimal calcularSubtotal(List<Producto> productos)
    BigDecimal aplicarDescuento(BigDecimal subtotal, BigDecimal porcentaje)
    BigDecimal calcularImpuesto(BigDecimal baseImponible)
    BigDecimal calcularTotal(List<Producto> productos, BigDecimal porcentaje)
    BigDecimal calcularTotal(List<Producto> productos) // sin cupón
}
```

Estas firmas son el **contrato**: tú decides la implementación, pero debe emerger de las pruebas, no escribirse por adelantado.

4. Preparación del entorno

1. Instala JDK 17 (o superior) y Maven. Verifica con `java -version` y `mvn -version`.
2. Crea un proyecto Maven (o usa el asistente de tu IDE) con esta estructura:

```
tdd-pedidos/
├── pom.xml
├── src/
│   ├── main/java/pe/edu/upn/pedidos/    ← Producto, CalculadoraPedido
│   └── test/java/pe/edu/upn/pedidos/     ← CalculadoraPedidoTest
```

3. Agrega JUnit 5 en el `pom.xml` y ejecuta `mvn test` (o Run desde IntelliJ IDEA, Eclipse o VS Code):

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.2</version>
  <scope>test</scope>
```

</dependency>

4. Sube el proyecto a un repositorio en GitHub: tdd-pedidos-grupoX.
5. Haz un commit al final de cada fase con prefijo: red: ..., green: ..., refactor: ... Este historial es la evidencia principal del proceso TDD.
6. Toma capturas de la ejecución de pruebas en cada fase Red (fallando) y Green (pasando).

5. Desarrollo: ciclos TDD

Desarrolla los ciclos en orden. No adelantes código de un ciclo posterior.

Ciclo 1 (ejemplo resuelto): subtotal de dos productos

● **Red** – Prueba escrita primero. El proyecto no compila: Producto y CalculadoraPedido no existen.

```
class CalculadoraPedidoTest {  
  
    private final CalculadoraPedido calc = new CalculadoraPedido();  
  
    private static BigDecimal soles(String monto) {  
        return new BigDecimal(monto);  
    }  
  
    private static Producto producto(String nombre, String precio, int cantidad) {  
        return new Producto(nombre, soles(precio), cantidad);  
    }  
  
    @Test  
    void subtotalDeDosProductosSumaSusPrecios() {  
        List<Producto> productos = List.of(  
            producto("Mouse", "50.00", 1),  
            producto("Teclado", "30.00", 1));  
        assertEquals(soles("80.00"), calc.calcularSubtotal(productos));  
    }  
}
```

● **Green** – Código mínimo (suma los precios; todavía ignora la cantidad):

```
public record Producto(String nombre, BigDecimal precio, int cantidad) { }
```

```
public class CalculadoraPedido {  
    public BigDecimal calcularSubtotal(List<Producto> productos) {  
        return productos.stream()  
            .map(Producto::precio)  
            .reduce(BigDecimal.ZERO, BigDecimal::add);  
    }  
}
```

● **Refactor** – Los métodos soles() y producto() ya evitan repetir new BigDecimal en cada prueba. Nada más por ahora.

El código es incompleto a propósito: no multiplica por la cantidad. La prueba pasa porque ambas cantidades son 1. El ciclo 2 lo pondrá en evidencia.

Ciclo 2: subtotal con cantidades y pedido vacío (RF1, RF5)

Elemento	Detalle
● Red	Escribe primero esta prueba y ejecútala. Debe fallar. Con el código del ciclo 1, la primera prueba devolverá 35.50 en lugar de 81.00. Observa también qué pasa con la lista vacía.
Nombre de la prueba	subtotalMultiplicaPrecioPorCantidad subtotalDeListaVacíaEsCero
Datos de entrada	Cuaderno S/ 25.50 × 2 · Lapicero S/ 10.00 × 3 List.of() (sin productos)
Resultado esperado	soles("81.00") soles("0.00")
● Green	Multiplica precio por BigDecimal.valueOf(cantidad) y devuelve el resultado con 2 decimales.
● Refactor	Si ves duplicación o nombres poco claros, mejóralos. Vuelve a ejecutar todas las pruebas: deben seguir en verde.

Pista: la lista vacía devuelve BigDecimal.ZERO, que tiene escala 0 ("0"), y la prueba espera "0.00". Es el primer encuentro con la escala de BigDecimal (ver la advertencia de la sección 3).

Ciclo 3: aplicar descuento (RF2)

Elemento	Detalle
● Red	Escribe primero esta prueba y ejecútala. Debe fallar. No compila: aplicarDescuento no existe.
Nombre de la prueba	aplicarDescuentoRestaElPorcentaje (usa @ParameterizedTest + @CsvSource)
Datos de entrada	Subtotal S/ 100.00 con 10% · con 0% · con 100%
Resultado esperado	S/ 90.00 · S/ 100.00 · S/ 0.00
● Green	Crea aplicarDescuento con el código mínimo para que pasen los tres casos.
● Refactor	Si ves duplicación o nombres poco claros, mejóralos. Vuelve a ejecutar todas las pruebas: deben seguir en verde.

```
// Pista: JUnit 5 convierte el texto del CSV a BigDecimal automáticamente
@ParameterizedTest(name = "{0} con {1}% -> {2}")
@CsvSource({ "100.00, 10, 90.00", "100.00, 0, 100.00", "100.00, 100, 0.00" })
void aplicarDescuentoRestaElPorcentaje(BigDecimal subtotal, BigDecimal porcentaje,
    BigDecimal esperado) { ... }
```

Ciclo 4: calcular IGV (RF3)

Elemento	Detalle
● Red	Escribe primero esta prueba y ejecútala. Debe fallar. No compila: calcularImpuesto no existe. Después, cuidado: 90.00 × 0.18 da 16.2000 y la prueba espera 16.20.
Nombre de la prueba	calcularImpuestoAplica18PorCiento (usa @ParameterizedTest)
Datos de entrada	Base imponible S/ 100.00 · S/ 90.00
Resultado esperado	IGV S/ 18.00 · S/ 16.20
● Green	Crea calcularImpuesto usando una constante para la tasa: new BigDecimal("0.18").
● Refactor	Si ves duplicación o nombres poco claros, mejóralos. Vuelve a ejecutar todas las pruebas: deben seguir en verde.

Ciclo 5: total del pedido y redondeo (RF4, RF5)

Elemento	Detalle
● Red	Escribe primero esta prueba y ejecútala. Debe fallar. La primera prueba no compila (calcularTotal no existe). La segunda fallará por centésimas si no redondeas.
Nombre de la prueba	totalConCuponDel10PorCientoEs106_20 totalRedondeaElIgvADosDecimales
Datos de entrada	1 producto de S/ 100.00, cupón 10% 3 lápices de S/ 11.11, sin cupón
Resultado esperado	S/ 106.20 S/ 39.33 (subtotal 33.33 + IGV 6.00)
● Green	Implementa calcularTotal combinando los métodos anteriores y aplica setScale(2, RoundingMode.HALF_UP).
● Refactor	Si ves duplicación o nombres poco claros, mejóralos. Vuelve a ejecutar todas las pruebas: deben seguir en verde.

¿Por qué 39.33? El IGV exacto de 33.33 es 5.9994. Sin redondear, el total sería 39.3294, un monto que no se puede cobrar. Esta prueba obliga a definir la regla de redondeo.

Ciclo 6: validaciones (RF6)

Elemento	Detalle
● Red	Escribe primero esta prueba y ejecútala. Debe fallar. El código actual no valida: devolverá montos negativos en lugar de lanzar la excepción.
Nombre de la prueba	precioNegativoLanzaExcepcion cantidadNoPositivaLanzaExcepcion (@ValueSource) descuentoFueraDeRangoLanzaExcepcion (@ValueSource)
Datos de entrada	Precio S/ -5.00 Cantidad 0 y -2 Descuento -1% y 101%
Resultado esperado	IllegalArgumentException en los tres casos. Para el precio, verifica también el mensaje: «El precio no puede ser negativo».
● Green	Agrega las validaciones al inicio de los métodos.
● Refactor	Si ves duplicación o nombres poco claros, mejóralos. Vuelve a ejecutar todas las pruebas: deben seguir en verde.

```
// Pista: assertThrows devuelve la excepción para revisar su mensaje
IllegalArgumentException ex = assertThrows(IllegalArgumentException.class,
    () -> calc.calcularSubtotal(productos));
assertEquals("El precio no puede ser negativo", ex.getMessage());
```

Observa que -1 / 0 / 100 / 101 son los valores límite del porcentaje de descuento que trabajaste en la Semana 08: 0 y 100 ya se probaron en el ciclo 3 como válidos.

Ciclo 7: refactorización final

Con todas las pruebas en verde, aplica al menos **dos** mejoras. Ejemplos:

- Extraer la validación a un método privado (validarProducto).
- Extraer el redondeo a un método privado (redondear) para no repetir setScale.
- Declarar TASA_IGV y CIEN como constantes static final.

- Agregar la sobrecarga `calcularTotal(productos)` sin cupón, que delega en la versión con porcentaje 0.

Después de cada cambio ejecuta todas las pruebas. Si algo falla, el refactor cambió el comportamiento: revierte y vuelve a intentarlo.

6. Bitácora de ciclos (completar)

Ciclo	Prueba escrita	Red: ¿por qué falló?	Green: ¿qué cambiaste?	Refactor realizado
1	<code>subtotalDeDosProductos...</code>	No compilaba: las clases no existían	Se creó <code>calcularSubtotal</code> sumando precios	Métodos <code>soles()</code> y <code>producto()</code> en la prueba
2				
3				
4				
5				
6				
7	—	—	—	

7. Preguntas de reflexión

1. En el ciclo 1 el código ignoraba la cantidad y aun así la prueba pasaba. ¿Qué te enseña esto sobre la elección de los datos de prueba?
2. ¿Por qué se usa `BigDecimal` y no `double` para montos de dinero? ¿Por qué se crea desde un `String` y no desde un `double`?
3. ¿Qué ocurrió con la escala de `BigDecimal` en los ciclos 2 y 4? ¿Qué alternativa a `assertEquals` existe para comparar solo el valor?
4. ¿Qué valores límite del porcentaje de descuento probaste y por qué esos?
5. La tienda decide que algunos productos estén exonerados de IGV. ¿Qué pruebas agregarías primero y qué parte del diseño cambiaría?

8. Entregables

- Enlace al repositorio con el proyecto Maven, las pruebas y el historial de commits por fase.
- Este documento completado: datos del grupo, bitácora, capturas de cada fase Red y Green, y respuestas de reflexión.
- Captura final de `mvn test` con todas las pruebas pasando (mínimo 15 casos de prueba, contando cada fila de `@CsvSource` y `@ValueSource`).

9. Rúbrica de evaluación

Criterio	Puntos	Se obtiene el puntaje completo si...
Pruebas unitarias	5	Cubren RF1 a RF6 con datos concretos, nombres descriptivos y uso adecuado de <code>@Test</code> , <code>@ParameterizedTest</code> y <code>assertThrows</code> .
Evidencia del ciclo TDD	5	El historial de commits muestra red → green → refactor en cada ciclo, con capturas de prueba fallando y pasando.
Código funcional	3	Todas las pruebas pasan, se usa <code>BigDecimal</code> y el redondeo es correcto.
Refactorización	3	Al menos dos mejoras aplicadas sin romper las pruebas y explicadas en la bitácora.
Bitácora y reflexión	4	Bitácora completa y respuestas que demuestran comprensión de TDD, de <code>BigDecimal</code> y de los valores límite.
Total	20	